# 物体计数


物体计数
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- 常见问题
  - 为什么我的识别效果不好?

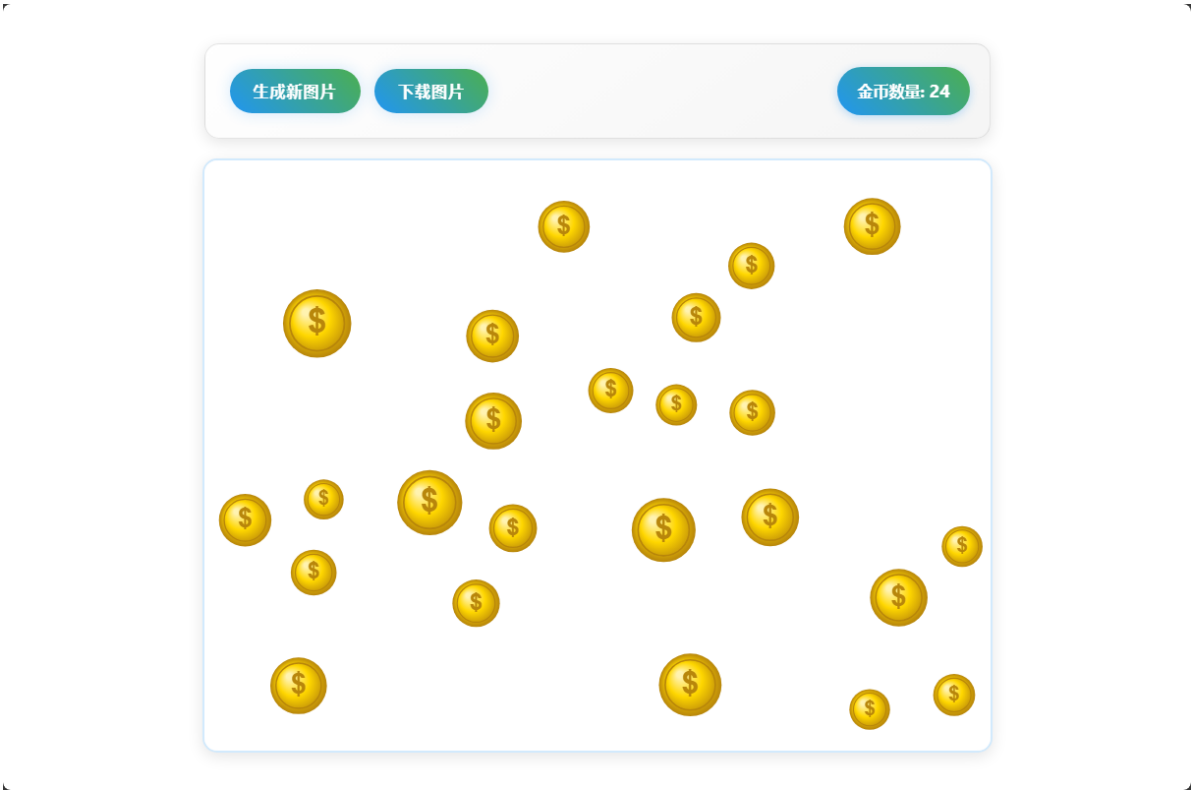

## 例程实验效果简介

基于上一节我们学习过的颜色识别，这一节我们来尝试统计颜色相近的物体个数

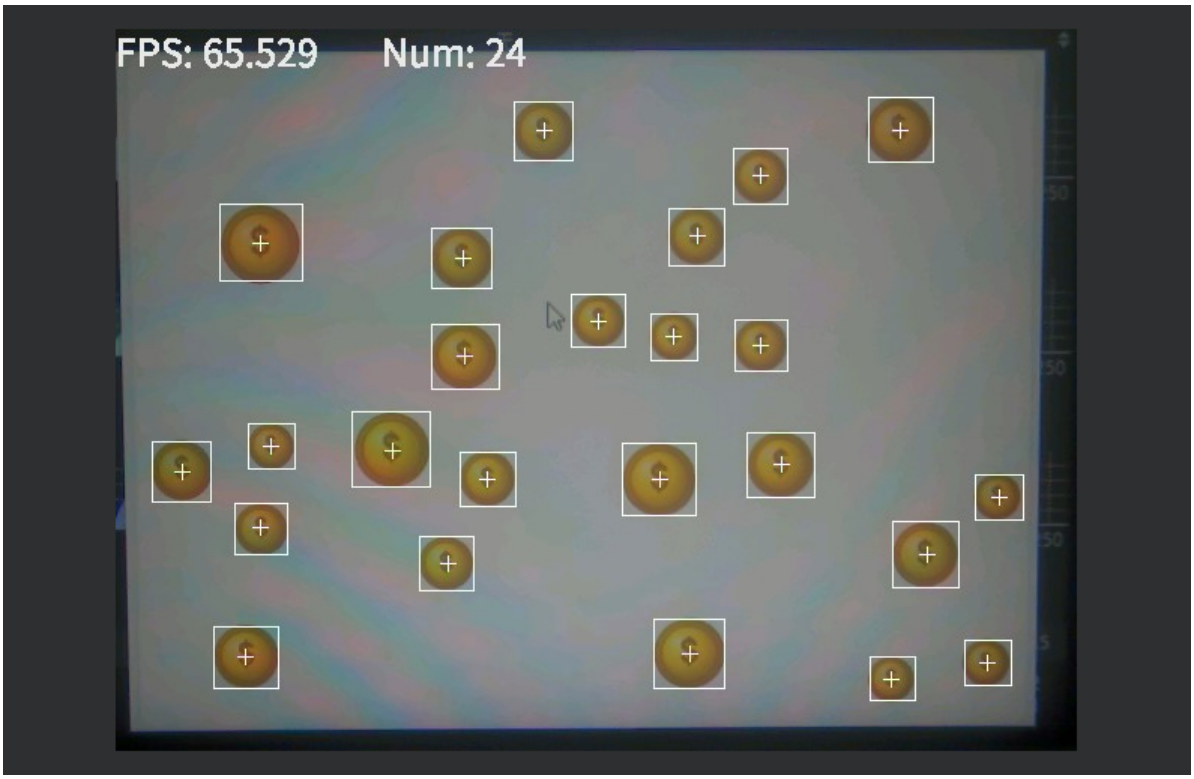
这里我们使用配套资料中的【金币生成】工具来生成随机数量的金币图片




生成一张如下图所示的图片：



然后我们运行这一节的例程代码，用摄像头对准这张图片，可以看到屏幕左上角识别出来了图片中有多少个金币




我们可以多生成几张图片测试一下，可以看到识别的速度还是很快的，准确率也比较高

# 代码讲解

本节例程代码文件位于【/ 源码汇总 / 05.Color / 02.item_count.py】

## 完整代码

```
import time
import os
import sys
from media.sensor import Sensor
from media.display import Display
from media.media import MediaManager

# 常量定义 / Constants definition
SCREEN_WIDTH = 640
SCREEN_HEIGHT = 480
# 待检测物体的LAB色彩空间阈值 / LAB color space thresholds
# 这里写的阈值是配套工具中【金币生成器】生成的金币的颜色
# Format: (L_min, L_max, A_min, A_max, B_min, B_max)
TRACK_THRESHOLD = [(0, 100, -7, 127, 10, 83)]
# 文字显示参数 / Text display parameters
FONT_SIZE = 25
TEXT_COLOR = (233, 233, 233)  # 白色 / White

def init_camera():
    """
    初始化并配置摄像头
    Initialize and configure the camera
    """
    # sensor = Sensor(width=1280,height=960)
```

```
    sensor = Sensor()
    sensor.reset()
    sensor.set_framesize(width=SCREEN_WIDTH, height=SCREEN_HEIGHT)
    sensor.set_pixformat(Sensor.RGB565)
    return sensor

def init_display():
    """
    初始化显示设备
    Initialize display device
    """
    # 初始化3.5寸MIPI屏幕和IDE显示
    # Initialize 3.5-inch MIPI screen and IDE display

 Display.init(Display.ST7701,width=SCREEN_WIDTH,height=SCREEN_HEIGHT,to_ide=True
)
    MediaManager.init()

def process_frame(img, threshold):
    """
    处理单帧图像，检测并标记目标物体
    Process single frame, detect and mark target objects

    Args:
        img: 输入图像 / Input image
        threshold: 颜色阈值 / Color threshold

    Returns:
        blobs: 检测到的物体列表 / List of detected objects
    """
    blobs = img.find_blobs([threshold])

    if blobs:
        for blob in blobs:
            # 绘制矩形框和中心十字 / Draw rectangle and center cross
            img.draw_rectangle(blob[0:4])
            img.draw_cross(blob[5], blob[6])

    return blobs

def draw_info(img, fps, num_objects):
    """
    在图像上绘制信息
    Draw information on image

    Args:
        img: 输入图像 / Input image
        fps: 帧率 / Frames per second
        num_objects: 检测到的物体数量 / Number of detected objects
    """
    info_text = f'FPS: {fps:.3f}       Num: {num_objects}'
    img.draw_string_advanced(0, 0, FONT_SIZE, info_text, color=TEXT_COLOR)

def main():
    """
    主程序入口
    Main program entry
    """
```

```
    # 初始化设备 / Initialize devices
    sensor = init_camera()
    init_display()
    sensor.run()

    # 创建时钟对象用于FPS计算 / Create clock object for FPS calculation
    clock = time.clock()

    try:
        while True:
            clock.tick()

            # 捕获图像 / Capture image
            img = sensor.snapshot()

            # 处理图像 / Process image
            blobs = process_frame(img, TRACK_THRESHOLD[0])

            # 显示信息 / Display information
            draw_info(img, clock.fps(), len(blobs))

            # 显示图像 / Show image
            Display.show_image(img)

            # 打印FPS / Print FPS
            print(f"FPS: {clock.fps():.3f}")

    except KeyboardInterrupt:
        print("Program terminated by user")
    except Exception as e:
        print(f"Error occurred: {str(e)}")
    finally:
        # 清理资源 / Cleanup resources
        sensor.deinit()
        Display.deinit()

if __name__ == "__main__":
    main()
```

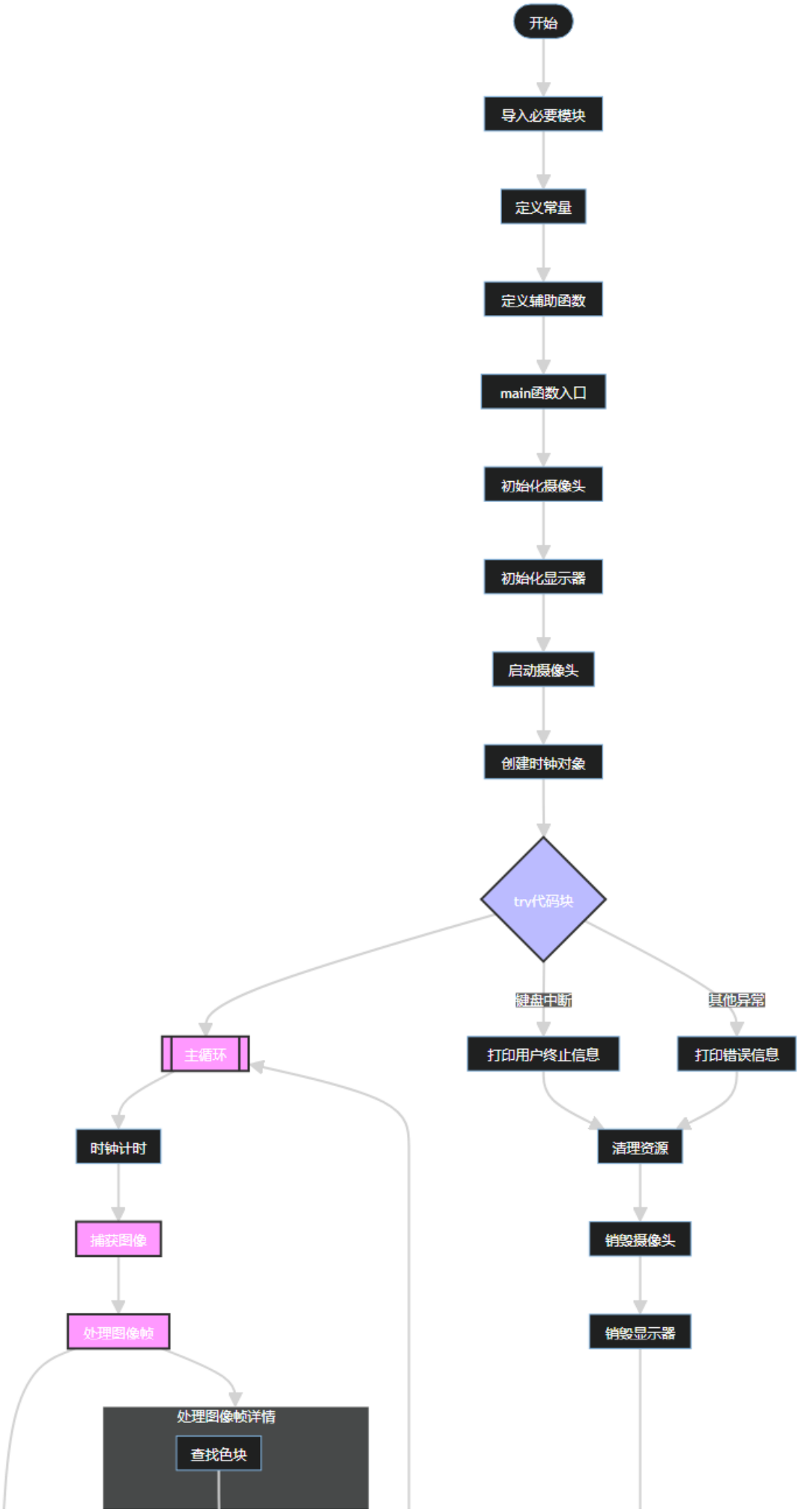
## 代码结构

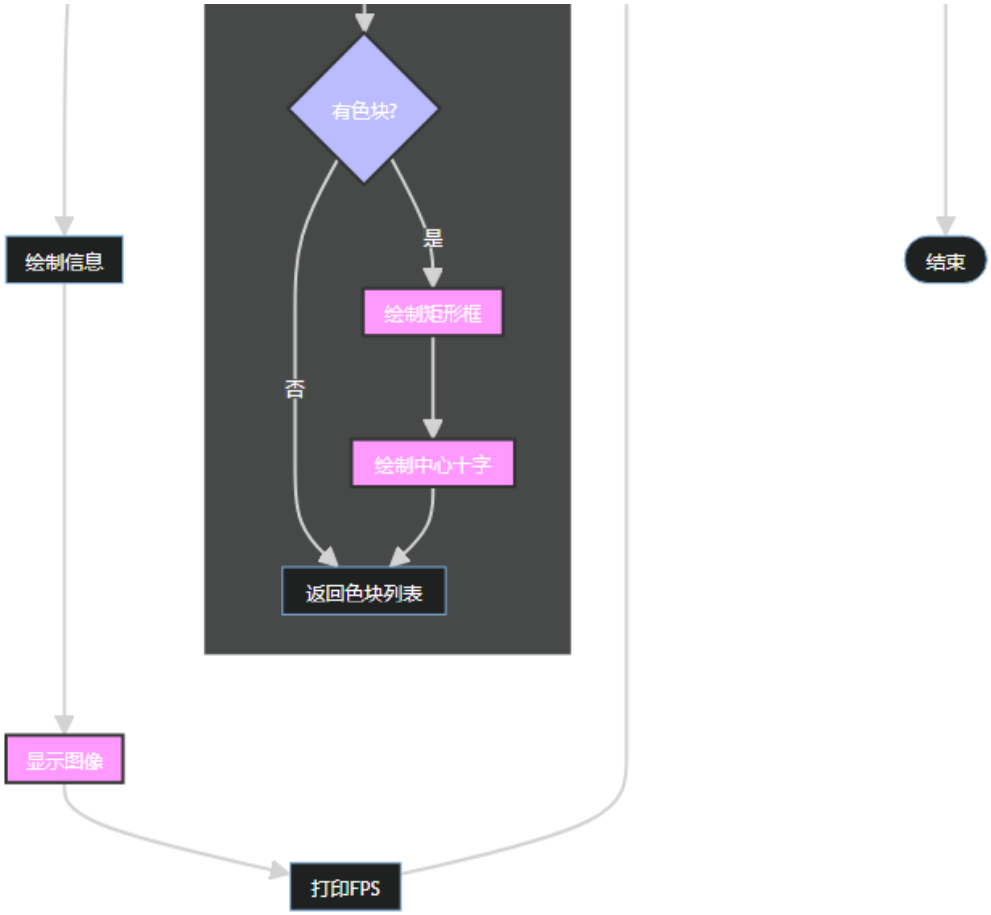
1. 初始化阶段:
    - 导入必要模块
    - 定义常量(屏幕尺寸、颜色阈值等)
    - 定义辅助函数
    - 初始化摄像头和显示器
    - 创建时钟对象
2. 主循环中:
    - 时钟计时
    - 捕获图像帧
    - 处理图像(查找色块、绘制标记)
    - 显示信息和图像
    - 打印FPS
3. 异常处理:

- 处理键盘中断
- 处理其他异常

4. 清理工作:
    - 销毁摄像头
    - 销毁显示器

详细流程如图所示:


开始
导入必要模块
定义常量
定义辅助函数
main函数入口
初始化摄像头
初始化显示器
启动摄像头
创建时钟对象
try代码块
键盘中断
其他异常
主循环
打印用户终止信息
打印错误信息
时钟计时
清理资源
捕获图像
销毁摄像头
处理图像帧
销毁显示器
处理图像帧详情
查找色块




# 常见问题

## 为什么我的识别效果不好?

由于光照、摄像头参数、屏幕参数的差异，例程代码中的阈值不一定是最合适的。我们可以使用前面教程中使用过的拍照例程代码，在真实的场景下，拍摄我们想要识别的物体。然后参考前面颜色识别教程中自定义颜色的过程，提取出更合适的颜色阈值